Dong Sub (Dave) Kim

dskim9218@gmail.com · +82 10-3693-6883 · Seoul, South Korea
Backend & Platform Engineer | Kotlin/Spring · Python/Django
dongsubkim.github.io

Summary

Backend engineer with 5+ years turning operations-floor requirements into data models and services, designed with maintainability and extensibility as first-order concerns. What I enjoy most is encoding complex business rules so the system makes the call: purchase orders and inbound receiving went from spreadsheets and phone calls to multi-center logistics, partner-booked inbound slots and backorders on virtual inventory, with the operations team's judgment turned into settings. Before building a feature I settle with product and operations whether it is needed and which structure fits; after an incident I trace the cause in logs and data and turn the fix into a shared rule so it does not recur.

Experience

Sendbird

September 2025 - Present

Software Engineer, Comms Applications (Backend)

Seoul

Customer-support ticketing product; the dashboard's authentication, permissions and account management; an internal operations back-office; the new chat platform's API gateway (Kotlin). Python/Django and Kotlin/Spring Cloud Gateway.

- **Omnichannel ticketing:** while maintaining the existing SDK flow where customers reach agents directly, built the handoff that turns AI-agent conversations on chat, email and voice into tickets for human agents on one platform; the voice handoff uses idempotent call sessions, a routing coordinator and event-driven webhook state transitions.
- **Third-party ticket migration:** owned the ticketing-side design and production execution of the company's first support-history migration from an external ticketing system — pre-migration of history, a short cut-over of inbound email routing, then delta re-collection by updated_at; idempotent migrators keyed on external IDs so a failed batch is fixed and rerun; per-batch validation scripts (a 6,988 vs 6,977 mismatch was traced to 11 delayed mappings, not loss). 181,140 tickets migrated and validated. Generalized into a reusable Zendesk → product migration engine (single orchestrator, chunk chaining within a 300-second limit, preflight checks, cross-organization email alias handling, reconcile for lost callbacks) and released it.
- **Zendesk-parity features** for customers moving off Zendesk, delivered alongside the migration engine for tickets, customers and agent accounts: time-based ticket automation on a Celery scheduler with lease recovery, parent-child tickets, and customizable manager reports — on a short timeline.
- **Database load incident → three-month query-optimization program** with the data-infrastructure team. A 502 for an 11,857-agent customer turned out to be serialization (59s), not SQL (6s): fixed in two steps (64s → 17s → 5.8s). Then owned the application side of the weekly program: dashboard list APIs moved to cursor pagination so counts disappear; unavoidable large-range counts moved to OpenSearch with a MySQL fallback for the 30-second sync lag; three proposed statistics-query rewrites benchmarked eight times on production data and rejected (1.4-6.4× slower), the real filter bug fixed instead; metrics collector 25 → 15 queries with results verified on 11 real-data cases across four regions; 2,763 OpenSearch timeouts split into three filter patterns, timeout cut to 3s and only the worst pattern routed to MySQL. The 502 was resolved the same week and the busiest region recorded lower DB usage.
- **Distributed rate limiting for the new chat API gateway** (Kotlin / Spring Cloud Gateway) on the rewrite of the legacy monolithic chat platform for AI agents: chose multi-window fixed-window counters (1s/60s, atomic per rule/scope in a small Lua script) over the legacy GCRA Lua path after benchmarking the 1:1 AI-agent traffic shape; default rules with per-customer custom rules stored in MySQL and propagated via Redis snapshot → pod-local cache with Pub/Sub invalidation plus revision polling; three blocking layers — pre-auth IP early deny, back-office manual blocking with audit trail, adaptive deny for callers that keep hitting the limit; PSK mutual authentication between the gateway and internal services, with the ticketing product accepting both legacy and new auth behind a flag so the cutover is decoupled from deploys. Decision logs shipped to Grafana/BigQuery with the data-platform team. In QA as of 2026-10.
- **Agent monitoring and statistics** over high-volume ticket data: statistics and summary APIs, report presets with asynchronous per-panel CSV ZIP export.

- **Platform and operations:** led the application side of moving an internal operations service from EC2 to Kubernetes (EKS) — parallel run against the same database, weighted DNS cutover, then legacy decommission; added OpenTelemetry tracing to the services I own; turned a mail-queue incident RCA into asynchronous parallel bulk sending and an outbound HTTP timeout standard so it cannot recur.
- **Team and reliability:** shipped the Jira OAuth integration solo within two months of joining; stacked-PR CI policy across four repos; production RCAs from data — a ticket-assignment-latency analysis that became an assigned-time sort feature, and a 1.98M-ticket first-assignment study used as the input for a queue-position feature.

Bucketplace (오늘의집)

September 2021 - September 2025

Software Engineer

Seoul

Seller (partner) accounts, purchase-order/inbound supply chain (SCM), order management and logistics — Kotlin / Spring microservices.

- Owned the SCM backend for four years; defined requirements and priorities directly with operations teams when no product owner was assigned.
- **Seller-account platform:** built a Kotlin/Spring account service replacing a Ruby on Rails monolith, to lift a one-account-per-partner limit that forced credential sharing and blocked permission separation: migrated ~40,000 accounts with a dual-write module reading old and new databases, JWT + Redis authentication with email/SMS verification, partner-consented OTP staff access, and shared Spring Cloud Gateway auth filters, with the core authentication team.
- **SCM from spreadsheets to a system, then multi-center:** built the purchase-order/inbound services that replaced manual, spreadsheet-driven processes; when a second warehouse arrived, replaced the hard-coded single center with a center domain model wired through every related entity (per-center stock for the same SKU, inter-center transfers, outbound-center selection) designed to absorb further centers.
- **Inbound scheduling self-service:** replaced phone-and-spreadsheet scheduling with partner-portal booking — suppliers pick an open inbound slot after receiving a purchase order and operations only approve; per-center slots and time windows are admin-configurable.
- **Backorders and virtual inventory:** proposed taking orders before stock arrives, using the delivery-date gap of furniture logistics and the inbound reservation data; started with a minimal scope to confirm operations could absorb it, then moved to virtual inventory where operations set, per SKU, what share of reserved inbound quantity may be sold; defined the escalation rules (emergency purchase orders, delivery-date changes) with the operations team.
- **SAP integration and redesign:** with no product owner, defined scope and priorities with operations; reworked the purchase-order model to the SAP purchase-order spec, extended the inventory model so center stock flows to both the commerce system and SAP, redesigned the purchase-order ERD and goods-receipt interface with other teams and external systems; migrated ~176,000 legacy logistics records to the new models.
- **Entity change history** kept out of the main transaction: snapshots collected by Hibernate hooks are written to MongoDB after the MySQL commit, with failures isolated and reported to Slack for manual replay.
- **Logistics operations:** replaced hard-coded order cutoffs with per-day configuration, supporting a cutoff change that reduced reported monthly contractor costs by ~KRW 15-20M; regional direct shipping cut the product damage rate 0.80% → 0.68% and delivery lead time 7.9 → 5.7 days; automated additional-delivery scheduling and capacity sync between TMS and OMS.
- **Engineering quality:** consolidated logistics/SCM repositories and entities; Detekt/ktlint/JaCoCo gates in CI and team conventions (814 code smells resolved; new purchase-order/inbound services at 87.6-95.9% coverage per repo); Spring Boot 2 / JDK 11 → Spring Boot 3 / JDK 21.

Naver Corporation

October 2020 - January 2021

Software Engineer Intern

- Processed 50,000-song lyric datasets with Pandas and trained multi-GPU PyTorch models for syllable-level song/lyric synchronization; maintained internal Python/Django analytics dashboards.

Basilica

November 2019 - April 2020

Machine Learning Engineer

- Designed and trained image/text classification models and embedding pipelines; built API endpoints for a browser-extension client.

Side Projects

MOGUM — filter-coffee brewing log (mogumcoffee.app, personal project)

- Connects recipes with the beans and gear you own, runs the brew, and records conditions and results for the next one. Running as a PWA; a React Native (Expo) app is planned.
- Sole owner from product design through the Kotlin/Spring Boot API (PostgreSQL, Spring Data JPA, Flyway, OAuth2/JWT), the Next.js web app, deployment and operations. TypeScript client generated from the OpenAPI contract; Testcontainers-based PostgreSQL integration tests cover authorization and data-preservation paths.
- Implementation is Codex-driven while product decisions, domain model, API contract and test criteria are mine; work is split across role-based subagents to keep context and token use small. Design examples: each brew stores an immutable snapshot of recipe revision, beans, gear and actual conditions, decoupled from later edits; the timer runs on a client clock and replays events with the same IDs and order after a temporary offline gap.

AI-Assisted Engineering

- Turned recurring work into reusable Claude Code / Codex skills and custom agents shared with the team: the release pipeline (tag → dev/official deploy workflows → deploy PRs → Slack review request), Jira ticket and worktree setup, PR authoring and review requests, daily work-evidence logs. Skills are kept working in both Claude Code and Codex.
- Used them for incident analysis: tooling that combines debug-pod execution, BigQuery, Grafana and Sentry to gather production evidence for RCAs (the 1.98M-ticket first-assignment study, an RDS memory-exhaustion RCA, the mail-queue incident).
- Spec-first pipeline with role-specific agents (requirements brief → design and design critique → implementation → tests and review → stacked-PR plan). Every agent result is checked against real system behavior before it is adopted, e.g. three proposed statistics-query rewrites rejected after benchmarking on production data.

Skills

- **Languages:** Kotlin, Java, Python, SQL, TypeScript
- **Frameworks:** Spring Boot, Spring Cloud Gateway, Spring Data JPA / Hibernate, Spring Batch, Django, Django REST Framework, Celery
- **Data:** MySQL, Redis / ElastiCache, MongoDB, OpenSearch / Elasticsearch, BigQuery, Kafka
- **Platform & Observability:** Kubernetes (EKS), Docker, Helm, ArgoCD, GitHub Actions CI/CD, AWS, OpenTelemetry, Prometheus / Grafana, Sentry
- **AI tooling:** Claude Code, Codex, MCP (Slack, Jira/Confluence, GitHub, BigQuery)

Education

University of California, Berkeley | Bachelor's Degree in Computer Science & Applied Mathematics

2012 - 2019 · GPA 3.80 / 4.00 (Major 3.88 / 4.00)